

EE4202 Database Systems

NOSQL DATABASES

NOSQL DATABASES

- What is NoSQL?



- To manage large amounts of data in organizations such as *Google, Amazon, Facebook, and Twitter* and in applications such as *social media, Web links, user profiles, marketing and sales, posts and tweets, road maps and spatial data, and e-mail.*
- *NoSQL databases are popular because of*
 - *Big data*
 - *Real-time web applications*

WHY NOSQL

- Relational databases (RDBMS) were created well before the internet, big data, and mobile communication became prominent
- RDMS were originally developed to be run in a single server. To increase the capacity of the database – need to upgrade the servers
- With the requirement of application change, no need to change the data model (schema-less)
- Flexible - Easy to handle semi-structured or unstructured data, (common in modern applications, especially those dealing with big data, IoT, and social media), can handle data in various formats
- Scalability- highly scalable, can handle large volumes of data by distributing data across multiple servers
- Horizontal Scaling- excel at horizontal scaling, can add more servers or nodes when requirements grow.
- Open Source and Cloud Integration

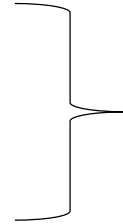
DISTRIBUTION MODELS

- There are two styles of distributing data:
- **Sharding** distributes different data across multiple servers, so each server acts as the single source for a subset of data.
- **Replication** copies data across multiple servers, so each bit of data can be found in multiple places.
- A system may use either or both techniques.
- Replication comes in two forms:
 - **Master-slave replication** makes one node the authoritative copy that handles writes while slaves synchronize with the master and may handle reads.
 - **Peer-to-peer replication** allows writes to any node; the nodes coordinate to synchronize their copies of the data.
- Master-slave replication reduces the chance of update conflicts but peer-to-peer replication avoids loading all writes onto a single point of failure.

DATA MODELS

Four types of data models exist as,

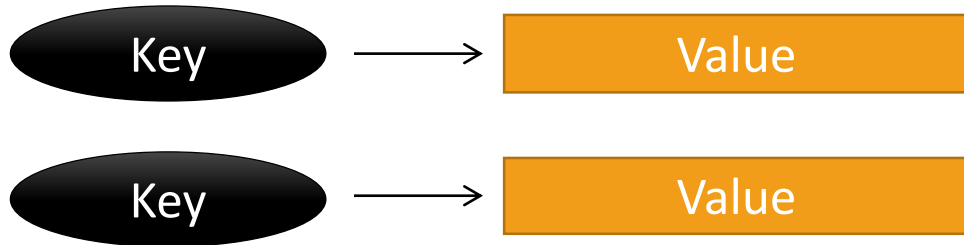
- 1) Key-value
- 2) Document
- 3) Column-Family stores
- 4) **Graph**



Aggregate Data models

KEY-VALUE DATABASES

- Key-value database store every single item in the database as a pair of attribute name (or 'key') together with its value
- It is a collection of key-value pairs.
- Key – must be unique
- New data can easily add as to the database as new key-value pairs.



Relational database	Key-value database(Riak)
Instance	Cluster
Table	Bucket
Row	Key-value
Row-id	key

Ex: Riak, Redis, Memcached DB, Project voldermort, Aerospike



redis



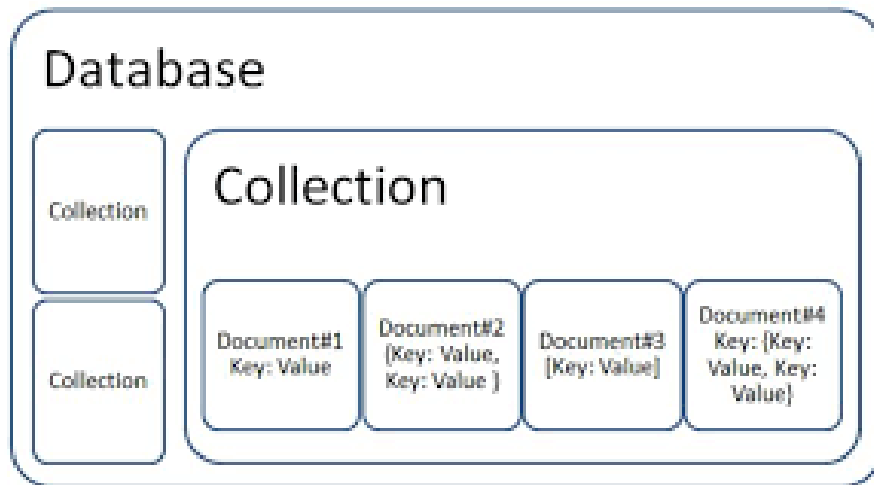
ORACLE®
NOSQL DATABASE

KEY-VALUE DATABASES

- It's like a primary index mapped to a ordered data file in relational databases.
- You can do only key lookup for the whole aggregate. You can't run a query nor retrieve part of aggregate.
- Suitable for storing session information, user profiles, shopping cart data.
- Limitations on value vary from software vendor to vendor.
- Not suitable for querying by data, multi operation transactions, relationship among data.
- Value is an object (aggregate) which stores all sorts of data.
- If necessary users can store data of a particular domain only in **domain buckets**.

DOCUMENT DATABASES

- Similar to key-value database, there is a key and a value which store in documents with hierarchical data structure
- Store data as collections of documents
- Documents can be specified in various formats, such as XML, JSON, etc.. Depending on the DBMS.



Relational database	Document database(MongoDB)
Instance	Instance
Table	Collection
Row	Document
Row-id	_id

Ex: MongoDB, ArangoDB, CouchDB, PostgreSQL, Jackrabbit, eXist

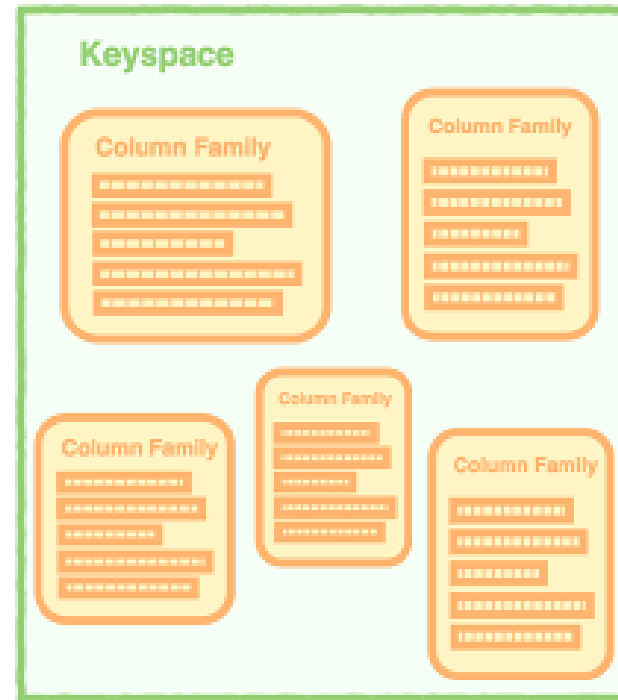


DOCUMENT DATABASES

- Looks up on aggregates based on using a query based on internal structure of document. Partial retrievals of aggregates are possible.
- The schema of data can differ among different documents.
- Try to ensure maximum availability using replica sets in master-slave mode.
- Transactions are only allowed at single document levels since commit and rollback functions are unavailable.
- Suitable use-cases are event logging (to store events), E-commerce applications like E-Bay, real time analytics – Easiness in document schema update unlike in RDBMS.
- Not suitable for transactions involving different documents.

COLUMN-FAMILY STORES

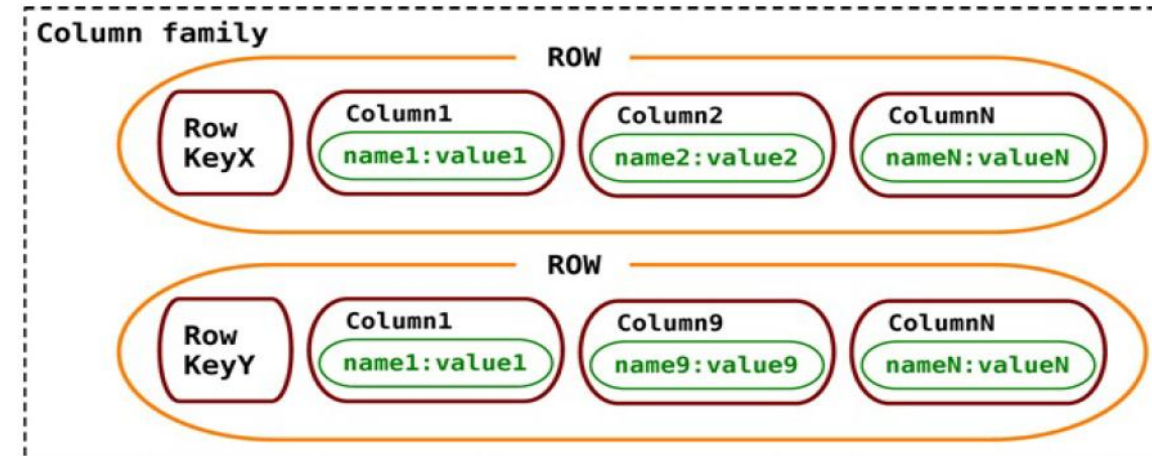
- Column-oriented model
- Store data in cells grouped in columns of data rather than as rows of data
- Keyspace – similar to schema in RDMS.
- Column-family – similar to tables in RDMS.



Relational database	Column-family database (Cassandra)
Instance	Cluster
Table	Column-family
Row	Row
Column	Column
Database	Keyspace

Source: (Pramod, Fowler)

Ex: Bigtable, Apache Cassandra, Apache Hbase, Hypertable



COLUMN-FAMILY STORES

- Row wise writing is efficient. Column wise reading is efficient. Stores groups of columns for all rows as the basic storage unit.
- Is a two-level aggregate structure. First key is the row identifier identifying the row aggregate. Second level values are referred to as columns. A column has a name-value pair. Column is identified using name.
- Note that each row can have different groups of columns unlike RDBMS.
- Transactions are atomic at row level.
- A set of columns can be nested inside value of another column to form a **super column**.
- Skinny rows have few columns. A wide row can have many columns. Use peer-peer replication.
- Ex: Blog entries, Event logging

GRAPH DATABASES

- Organize data in the form of graphs, which is a collections of nodes and edges
- Node – an entity, edge- connection/relationship between two entities.

Relational database	Graph database(Neo4J)
Relationship	Directional edge
Table	Graph
Row	Node

Ex: GraphDB, Neo4J, Sparksee, RedisGraph, OrientDB



GRAPH DATABASES

- Support ACID (atomicity, consistency, isolation, and durability) transactions.
- Aggregate data models have large records with simple connections.
- Graph databases have *small records with complex interactions* between the records.
- It is a data structure of nodes connected by edges.
- Ideal for capturing data consisting of complex relationships. Relationships can have properties.
- Compared to relationships in relational databases, it takes less time in graph databases to navigate and retrieve using queries. But, data insertion time is high.
- Uses master-slave architecture for reading and writing. Reading can be done from any node. Writing must always acknowledge the master. Sharding is difficult in graph databases.
- Adding or removing any node or edge must occur as a transaction.
- Ex: Social networks, location-based services (Google Maps), recommendation engines.

AGGREGATE DATA MODELS

- In relational model, a tuple consists of atomic and single valued fields.
- All database retrievals returns a set of tuples.
- In aggregate data models, nested tuples are allowed. It can contain complex records(tuples) which consists of other records or lists inside it.
- Aggregates make it easier for database to manage storage over clusters.
- Aggregate-oriented databases work best when most data interaction is done with the same aggregate; aggregate-ignorant databases are better when interactions use data organized in many different formations.
- Aggregates can be indexed by a key used to lookup those aggregates
- Do not support ACID (Atomic, consistent, Isolated, Durable) transactions between aggregates unlike relational databases.
- Transactions are done on a single aggregate usually. Aggregate is the boundary for an ACID operation.

CONSISTENCY

- Write-write conflicts occur when two clients try to write the same data at the same time. Read-write conflicts occur when one client reads inconsistent data in the middle of another client's write.
- Pessimistic approaches lock data records to prevent conflicts. Optimistic approaches detect conflicts and fix them.
- **Consistency** means agreement between database values and retrievals/updates or between replicas of data.
- Distributed systems see read-write conflicts due to some nodes having received updates while other nodes have not. **Eventual consistency** means that at some point the system will become consistent once all the writes have propagated to all the nodes.
- A client can write and then immediately read the new value. This can be difficult if the read and the write happen on different nodes.

CAP THEOREM

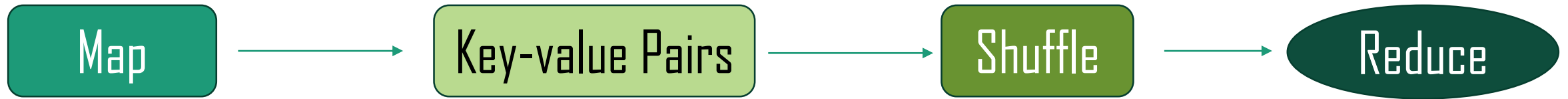
- To get good consistency, you need to involve many nodes in data operations, but this increases latency. So you often have to trade off consistency versus latency.
- Latency is the delay in communication from one end to another end.
- Relational databases have strong consistency due to single server architecture.
- **CAP** stands for Consistency, Availability and Partition tolerance.
- **Availability** stands for ability to read and write data at a given time based on the availability of nodes.
- **Partition tolerance** means that the cluster can survive communication breakages in the cluster that separate the cluster into multiple partitions unable to communicate with each other.
- CAP theorem states that it is not possible to guarantee all three of the desirable properties—consistency, availability, and partition tolerance—at the same time in a distributed system with data replication
- In other words, The CAP theorem states that if the network partition tolerant (high replication), you have high availability of data but less consistency.

VERSION STAMPS & TRANSACTIONS

- Version stamp is a field that changes every time the underlying data in the record changes. When you read the data; you keep a note of the version stamp, so that when you write data you can check to see if the version has changed.
- Version stamps help you detect concurrency conflicts. When you read data, then update it, you can check the version stamp to ensure nobody updated the data between your read and write.
- Version stamps can be implemented using counters, GUIDs, content hashes (Hash value of a particular record), timestamps (problematic when time is not synchronized), or a combination of these.
- GUID (Globally Unique Identifier), a large (16 byte) random number that's guaranteed to be unique.
- With distributed systems, a vector of version stamps allows you to detect when different nodes have conflicting updates. A node will update its version stamp in the vector when a write operation is done. When two nodes communication these version vectors will be checked.

MAP REDUCTION

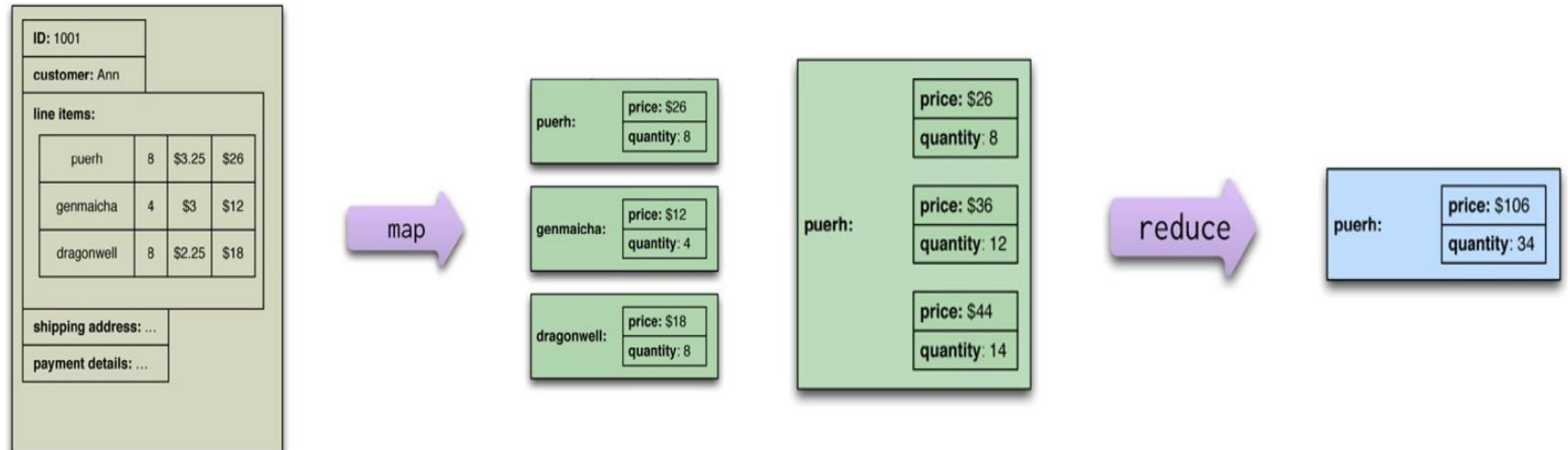
- Map reduction is a way to group all the data based on a key value using the map function and perform operations on this grouped data using the reduce function.
- Map-reduce is a pattern to allow computations to be parallelized over a cluster.



- The **map** task reads data from an aggregate and breaks it down to relevant key-value pairs. Maps only read a single record at a time and can thus be parallelized and run on the node that stores the record.
- **Reduce** tasks take many values for a single key output from map tasks and summarize them into a single output. Each reducer operates on the result of a single key, so it can be parallelized by key.

MAP REDUCTION

- Reducers that have the same form for input and output can be combined into pipelines. This improves parallelism and reduces the amount of data to be transferred.
- Map-reduce operations can be composed into pipelines where the output of one reduce is the input to another operation's map.
- If the result of a map-reduce computation is widely used, it can be stored as a materialized view.



References

- [1] Pramod J. Sadalage and Martin Fowler, “NoSQL distilled”, ISBN-10:0321826620